

## โปรแกรม: Real-time CDC Broker System

### 1. วัตถุประสงค์ของโปรแกรม

โปรแกรมนี้อธิบายระบบเครือข่ายที่ใช้สถาปัตยกรรมแบบ Publisher/Subscriber เพื่อจำลองการทำงานของระบบ Change Data Capture (CDC) ใช้สำหรับส่งต่อข้อมูลที่มีการเปลี่ยนแปลงในฐานข้อมูล (เช่น INSERT, UPDATE, DELETE) จากระบบต้นทาง (Publisher) ไปยังแอปพลิเคชันปลายทาง (Subscriber) หลายๆ ตัว พร้อมกันแบบเรียลไทม์ เพื่อให้ระบบต่างๆ สามารถนำข้อมูลไปประมวลผลหรือทำงานต่อได้ทันที

### 2. คุณลักษณะของแอปพลิเคชัน (Application Characteristics)

- Publisher-Subscriber Architecture: แอปพลิเคชันใช้รูปแบบโครงสร้างที่แยกผู้ส่ง (Publisher) และผู้รับ (Subscriber) ออกจากกัน โดยมี Server ทำหน้าที่เป็น Broker กลางในการจัดการข้อความ
- Real-time & Event-Driven: ข้อมูลถูกส่งออกทันทีที่มีเหตุการณ์เกิดขึ้น (Push-based) โดยผู้รับไม่ต้องส่ง Request ไปถามซ้ำๆ (No Polling)
- One-to-Many Distribution: รองรับการเชื่อมต่อจาก Subscriber หลายเครื่องพร้อมกัน โดยสามารถเลือกติดตาม (Subscribe) เฉพาะตารางข้อมูลที่น่าสนใจได้
- Persistent Connection: ผู้รับและผู้ส่งจะคงการเชื่อมต่อ (Keep-alive) ไว้ตลอดเวลาเพื่อรอรับ/ส่งสตรีมข้อมูล
- Data Serialization: โครงสร้างข้อมูลที่ส่งใน Payload จะถูกแปลงเป็นรูปแบบ JSON เพื่อให้รองรับโครงสร้างข้อมูลที่ซับซ้อน (Nested Data)

### 3. รูปแบบบริการของ Transport Layer ที่ใช้

แอปพลิเคชันนี้เลือกใช้ TCP (Transmission Control Protocol) ด้วยเหตุผลดังนี้:

- Reliability: ข้อมูล CDC เช่น คำสั่งลบ (DELETE) หรือคำสั่งแก้ไข (UPDATE) ข้อมูลทางการเงิน หากเกิดการสูญหายระหว่างทาง (Packet Loss) จะทำให้ฐานข้อมูลปลายทางพังและข้อมูลไม่ตรงกับความจริง TCP มีระบบ Retransmission ที่รับประกันว่าข้อมูลจะส่งถึงปลายทางครบถ้วน
- In-order Delivery: ลำดับเวลาเป็นสิ่งสำคัญมาก หากเหตุการณ์ "INSERT" ถูกส่งไปถึงช้ากว่า "DELETE" ระบบปลายทางจะเกิดข้อผิดพลาด TCP รับประกันว่าสตรีมข้อมูลจะถูกเรียงลำดับอย่างถูกต้องเสมอ
- Data Integrity: ข้อมูลมีขนาดใหญ่และซ้อนกันเป็น JSON การทำงานของ TCP ช่วยจัดการการแบ่งและประกอบ Packet ให้สมบูรณ์โดยที่แอปพลิเคชันไม่ต้องจัดการเอง

## การออกแบบ Application-Layer Protocol

### ชื่อโปรโตคอล: CDCP (Change Data Capture Protocol)

โปรโตคอลถูกออกแบบโดยใช้รูปแบบ Text-based (คล้าย HTTP) โดยมีตัวแบ่งส่วนระหว่าง Header และ Payload คือ `\r\n\r\n`

### 1. Request Messages (ฝั่ง Client ส่งไป Server)

#### 1.1 คำสั่ง Subscribe (สำหรับผู้รับข้อมูล)

- รูปแบบ: `SUBSCRIBE [TABLE_NAME]\r\n\r\n`
- คำอธิบาย: Client ส่งเพื่อขอขึ้นทะเบียนรับข้อมูลจากตารางที่กำหนด
- ตัวอย่าง: `SUBSCRIBE orders_table\r\n\r\n`

#### 1.2 คำสั่ง Publish (สำหรับผู้ส่งข้อมูล)

- รูปแบบ: `PUBLISH [TABLE_NAME]  
[OPERATION]\r\n\r\n[JSON_PAYLOAD]`
- คำอธิบาย: Client ส่งเหตุการณ์พร้อมข้อมูลแนบไปยัง Broker

### 2. Response & Event Messages (ฝั่ง Server ส่งไป Client)

#### 2.1 Status Response (การตอบกลับสถานะการทำงาน)

- รูปแบบ: `CDCP/1.0 [STATUS_CODE]  
[STATUS_PHRASE]\r\n\r\n`
- ตัวอย่าง
  - `CDCP/1.0 200 SUBSCRIBED\r\n\r\n` (ติดตามสำเร็จ)
  - `CDCP/1.0 201 PUBLISHED\r\n\r\n` (รับข้อมูลจาก Publisher สำเร็จ)
  - `CDCP/1.0 400 BAD_REQUEST\r\n\r\n` (ส่งคำสั่งผิดรูปแบบ)

#### 2.2 Event Message (การกระจายข้อมูลไปยัง Subscriber)

- รูปแบบ: `EVENT [TABLE_NAME]  
[OPERATION]\r\n\r\n[JSON_PAYLOAD]`

### 3. ลำดับการสื่อสาร (Message Sequence Flow)

- Subscriber -> Broker: ส่งคำสั่ง `SUBSCRIBE users_table\r\n\r\n`

- Broker -> Subscriber: ตอบสถานะ CDCP/1.0 200  
SUBSCRIBED\r\n\r\n
- Publisher -> Broker: ตรวจพบเหตุการณ์ ส่ง PUBLISH users\_table  
INSERT\r\n\r\n{"id": 1}
- Broker -> Publisher: ตอบสถานะรับมอบ CDCP/1.0 201  
PUBLISHED\r\n\r\n
- Broker -> Subscriber: กระจายข้อมูล EVENT users\_table  
INSERT\r\n\r\n{"id": 1}

#### 4. ข้อจำกัดของระบบและการรับมือข้อผิดพลาด

- Maximum Buffer Size: ระบบกำหนดขนาดการอ่านข้อมูล (Receive Buffer) ไว้ที่สูงสุด 4,096 Bytes ต่อ 1 รอบคำสั่ง recv() หากข้อความ Event หรือ JSON Payload มีขนาดใหญ่เกินกว่านี้ ข้อมูลจะถูกระบบหั่นเป็นส่วนๆ ตามธรรมชาติของสตรีม TCP ผู้รับจะต้องอาศัยสัญลักษณ์ \r\n\r\n และปีกกาของ JSON เพื่อตรวจสอบความสมบูรณ์และประกอบข้อมูลกลับด้วยตนเอง
- Single Point of Failure : ตามสถาปัตยกรรม Pub/Sub หากตัว Server (Broker) เกิดข้อผิดพลาด หรือหยุดการทำงานกะทันหัน ระบบเครือข่ายทั้งหมดจะถูกตัดขาดทันที Publisher จะไม่สามารถส่งข้อมูลได้ และ Subscriber จะไม่ได้รับ Event ใดๆ
- Orphaned Socket Handling: มีการเขียนดักจับ ConnectionResetError ไว้ที่ฝั่ง Server เพื่อรับมือกับกรณีที่ Subscriber ปิดโปรแกรมกะทันหัน ไฟดับ หรือสายแลนหลุด ระบบจะลบ Socket ของ Client นั้นออกจาก Dictionary การติดตามตารางทันที เพื่อป้องกันปัญหา Memory Leak และลดการสูญเสีย Bandwidth ในการ Broadcast ข้อมูลไปหาปลายทางที่ตายไปแล้ว
- Force Data Persistence: ในฝั่งของ Subscriber เมื่อได้รับข้อมูลแล้ว จะถูกบังคับให้เขียนลง ไฟล์และสั่ง f.flush() ลงฮาร์ดดิสก์แบบบรรทัดต่อบรรทัดทันที เพื่อลดความเสี่ยงที่ข้อมูล CDC ใน หน่วยความจำชั่วคราว (RAM) จะสูญหายหากแอปพลิเคชันฝั่งผู้รับล่มกะทันหัน